HOME LAB WALKTHROUGH

Active Directory Attack & Defense Lab

LLMNR / NBT-NS / mDNS credential-poisoning attack, hash cracking, and a full defensive fix — built and tested end to end on an isolated VirtualBox lab (Kali Linux, Windows Server 2022, Windows 11).

Prepared by: Anush

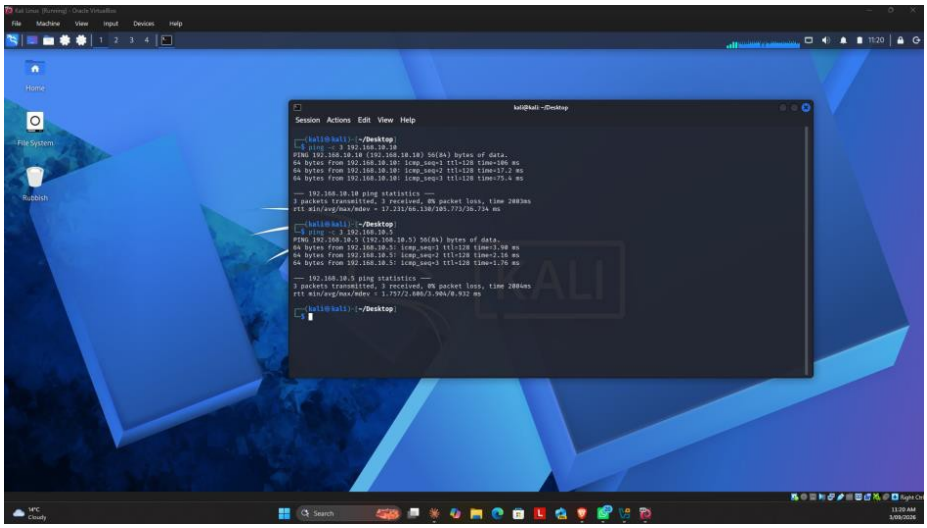
Type: Personal home lab project (not professional/client work)

Status: Studying toward CompTIA Security+

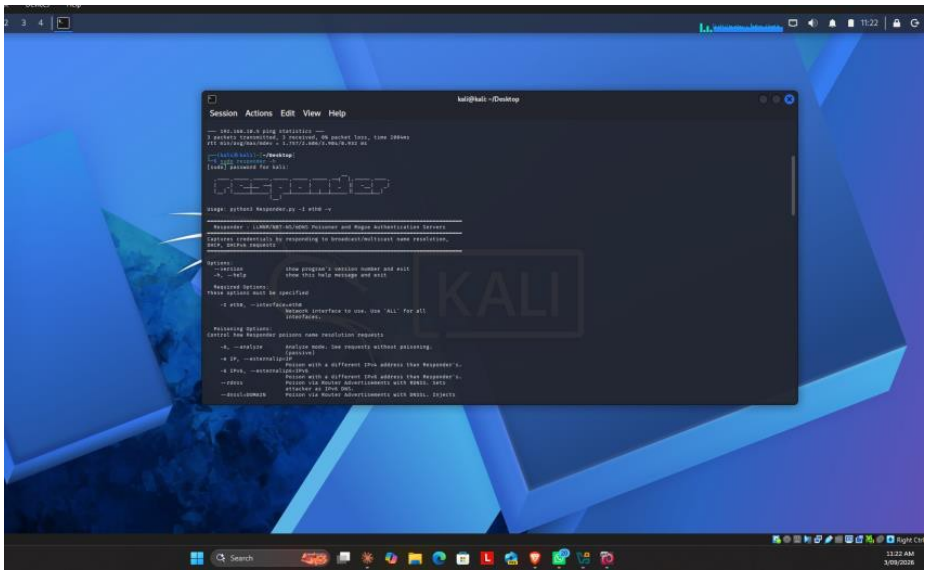
Phase 1 · Lab Setup & Reconnaissance

Confirming all three lab machines are up, reachable, and that the attack tool is ready before touching anything.

1. Powered on all three lab VMs — Kali Linux, Windows Server 2022 (the domain controller), and Windows 11 (the domain-joined client) — then confirmed basic network connectivity from Kali by pinging both Windows machines. Both replied with 0% packet loss, confirming all three machines could see each other on the LabNet subnet before starting anything.



2. Confirmed Responder (an LLMNR/NBT-NS/mDNS poisoning tool) was already installed on Kali by running responder -h. Responder is an open-source penetration testing tool that listens on a local network and answers name-resolution broadcast requests sent out by Windows machines, pretending to be whatever host was being searched for — which tricks the victim machine into sending its login credentials straight to the attacker.

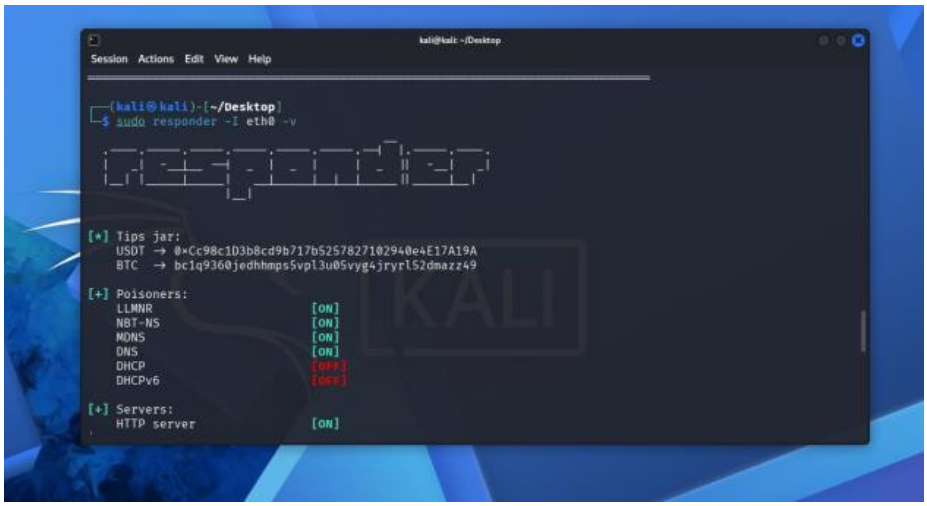


> **📌 What is Responder, in plain terms?**
>
> Formal definition: Responder is a tool that listens for Windows name-resolution broadcast requests (LLMNR, NBT-NS, mDNS) and answers them itself, impersonating whatever host the victim was trying to reach, in order to capture the authentication credentials sent in response. Simple example: imagine you shout "Does anyone know where Dave sits?" across an office because you don't know his desk number. Normally nobody who isn't actually named Dave would answer. Responder is like a stranger who shouts back "I'm Dave, come on over!" — and if you then hand him something meant for Dave (like your login details), he now has it instead.

Phase 2 · Executing the LLMNR Poisoning Attack

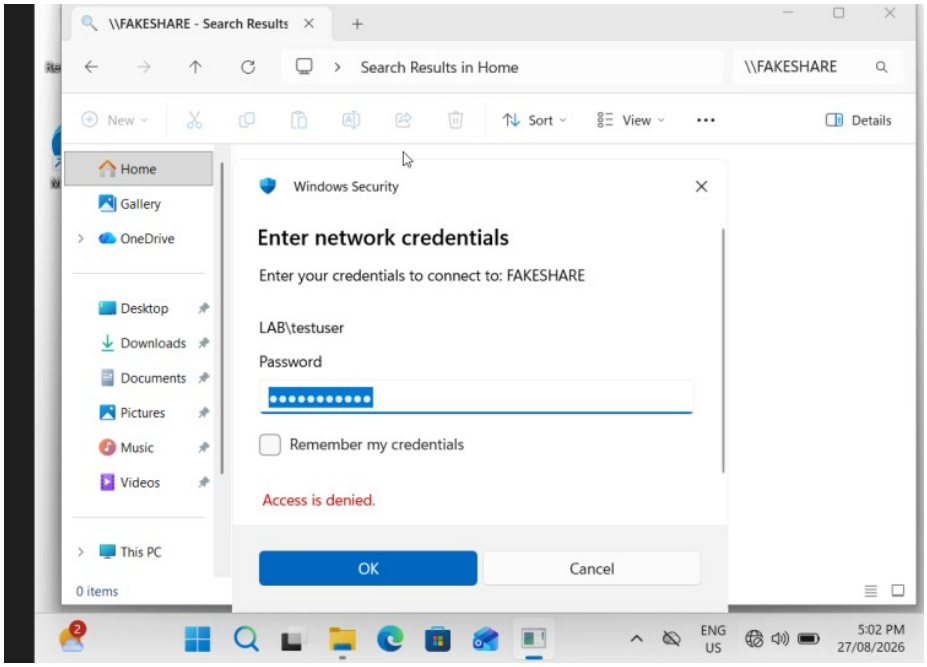
Putting Responder into listening mode, then tricking Windows 11 into broadcasting a name-resolution request it can intercept.

3. Started Responder in listening mode on Kali's network interface (sudo responder -I eth0 -v), with LLMNR, NBT-NS, and mDNS poisoning all shown as ON by default. Responder is now silently waiting on the LabNet subnet for any Windows machine to broadcast a name it doesn't recognize.

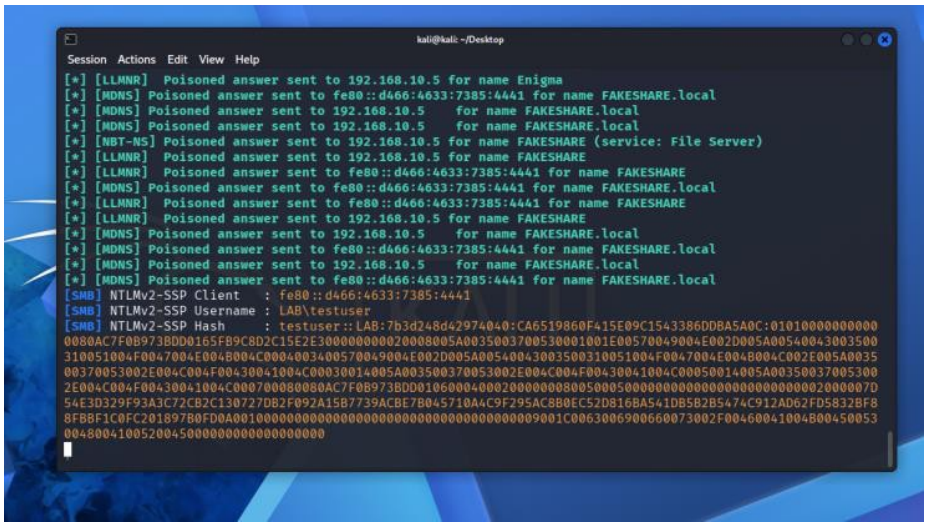


> **📌 What is LLMNR?**
> LLMNR (Link-Local Multicast Name Resolution) is a backup way for Windows computers on the same local network to find each other by name when normal DNS fails — by shouting the request out to every device nearby instead of asking a proper server. That "shouting to everyone" behavior is exactly the weakness Responder exploits: anyone listening can simply answer first and pretend to be the machine being searched for.

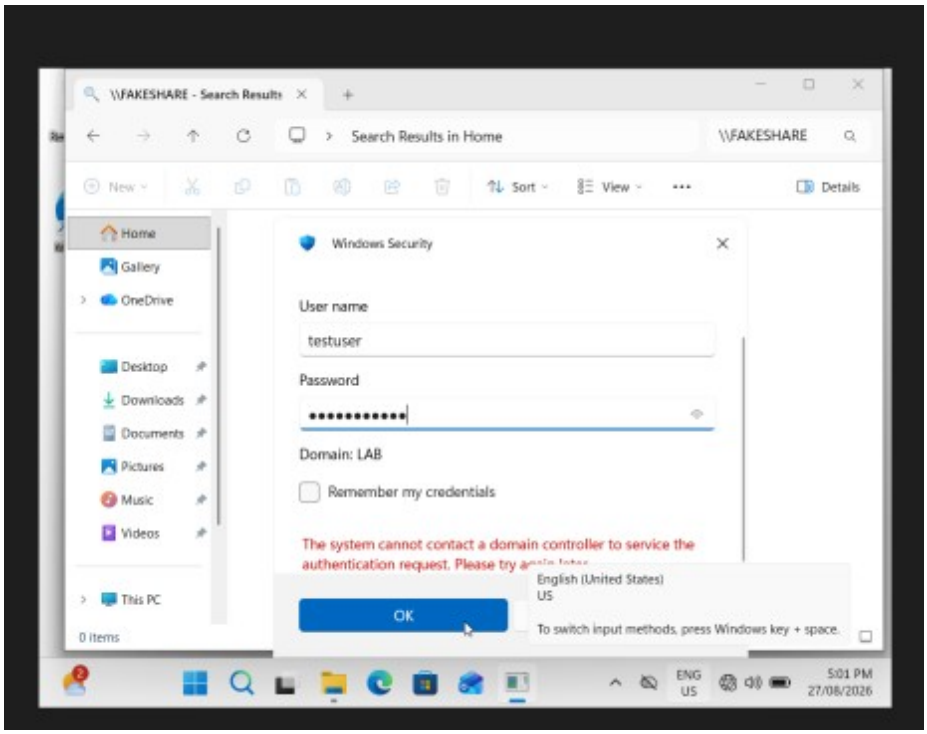
4. Triggered a failed network lookup on Windows 11 by opening the Run dialog (Win+R) and typing \\FAKESHARE — a network share name that doesn't actually exist anywhere on the network. Because normal DNS has no record of "FAKESHARE," Windows 11 automatically fell back to broadcasting an LLMNR request, asking the whole local network "does anyone know a host called FAKESHARE?" — exactly the kind of broadcast Responder was waiting for.



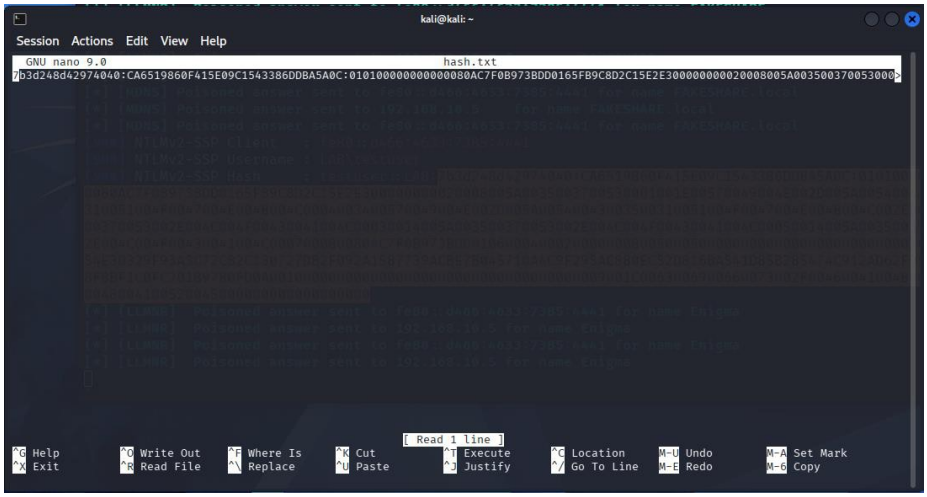
5. Entered fake test credentials (testuser / a made-up password) into the Windows Security prompt that appeared for \\ FAKESHARE, and clicked OK. Windows 11 immediately showed "Access is denied" — but that failure didn't matter. By the time this login attempt was submitted, Windows had already tried to authenticate against Responder's fake FAKESHARE host, and in doing so it sent an NTLMv2 authentication hash across the network in the process.



6. Back on the Kali terminal, Responder — impersonating FAKESHARE — intercepted and displayed the full NTLMv2-SSP hash in real time, tagged with the domain and username used: LAB\testuser. Checking Windows 11 again afterward showed a second, different error ("the system cannot contact a domain controller") since testuser was never a real domain account. Both Windows-side failures were expected and didn't matter — the actual goal of the attack, capturing a password hash in transit, had already succeeded the moment Windows attempted authentication.



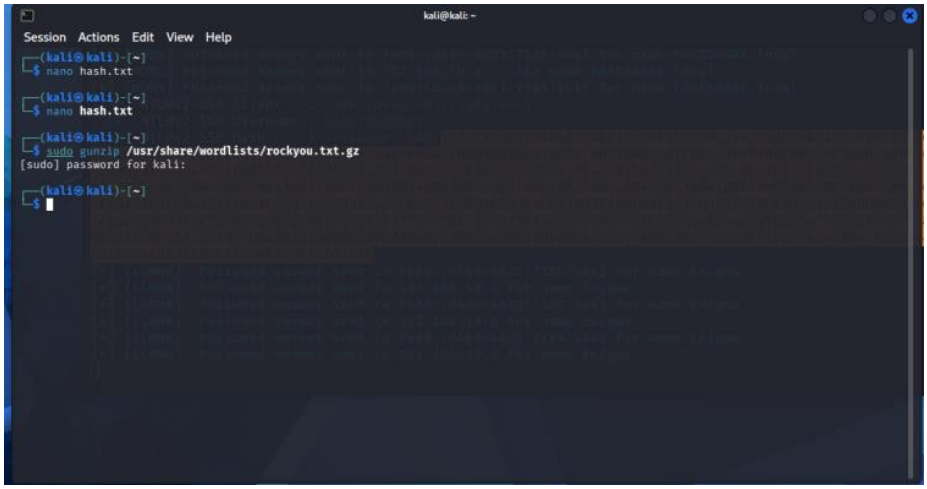
7. Saved the captured NTLMv2 hash into a text file (hash.txt) using nano, to prepare it for offline password cracking with Hashcat.



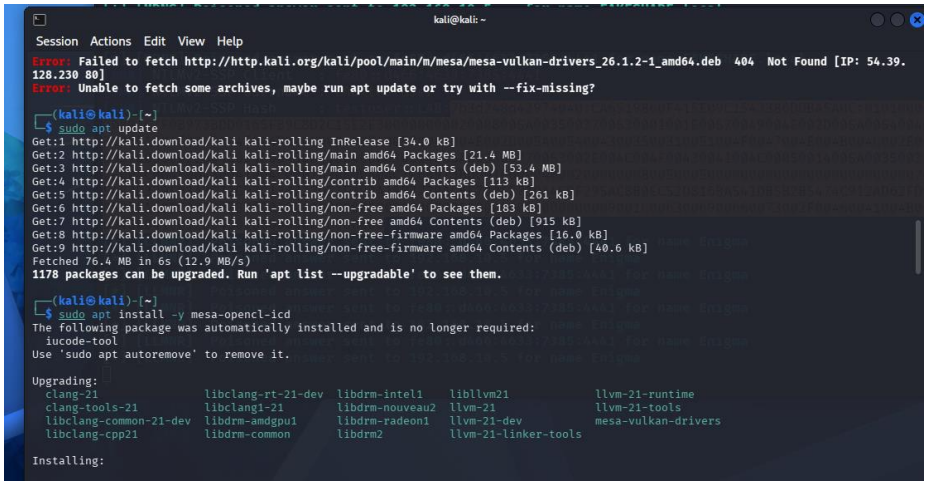
Phase 3 · Cracking the Captured Hash

Turning the captured hash into a plaintext password — including a real GPU-driver dead end that forced a change of tools.

8. Unzipped Kali's built-in rockyou.txt wordlist (a list of roughly 14 million real leaked passwords) to use as the dictionary for the cracking attempt.

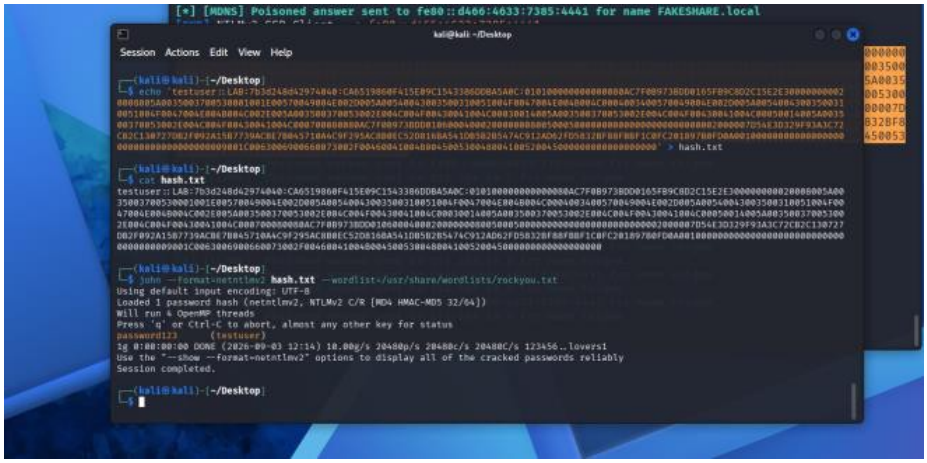


9. First attempt to crack the hash with Hashcat failed immediately: "No OpenCL, HIP or CUDA compatible platform found." Hashcat expects a GPU driver to run its cracking engine on, and this VirtualBox VM has no real graphics card passed through to it. Tried installing a CPU-based OpenCL driver (pocl-opencl-icd) as a workaround, but that exact package wasn't available in Kali's repository. Searched for an alternative and found mesa-opencl-icd, a software-based OpenCL driver that doesn't need a real GPU. The first install attempt failed with a 404 error caused by a temporarily out-of-sync package mirror; running sudo apt update refreshed the package list, and the second install attempt succeeded.



> **📌 Why not just keep pushing on Hashcat?**
>
> Even after installing mesa-opencl-icd, Hashcat still reported "No devices found" — the software driver wasn't exposing a usable compute device inside this particular VM. Rather than keep fighting GPU driver plumbing for the sake of cracking a single hash, the simpler move was to switch to John the Ripper, which cracks hashes purely on the CPU with no graphics driver dependency at all. This is a completely normal, realistic call to make — knowing when to stop fighting a tool and switch approaches is itself part of the skill.

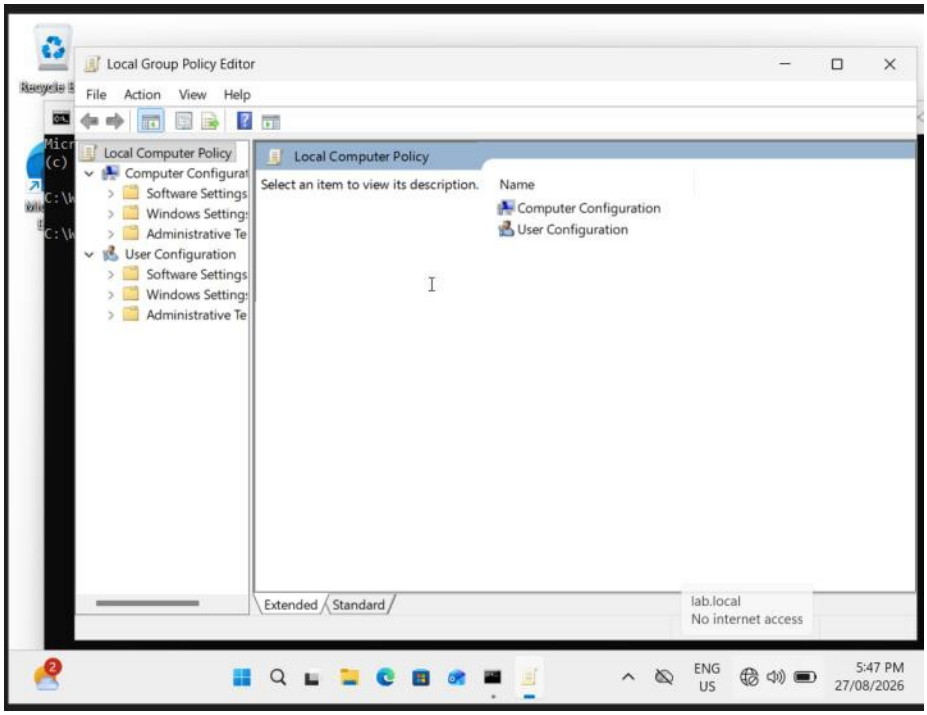
10. Before John the Ripper could run, the hash file needed a small fix — an earlier copy-paste had accidentally cut off the username and domain portion of the hash, leaving John unable to read it. Once the full hash (testuser::LAB:...) was back in place, John the Ripper cracked it in under one second against the rockyou.txt wordlist, recovering the plaintext password password123 — completing the full chain from network poisoning to plaintext password.



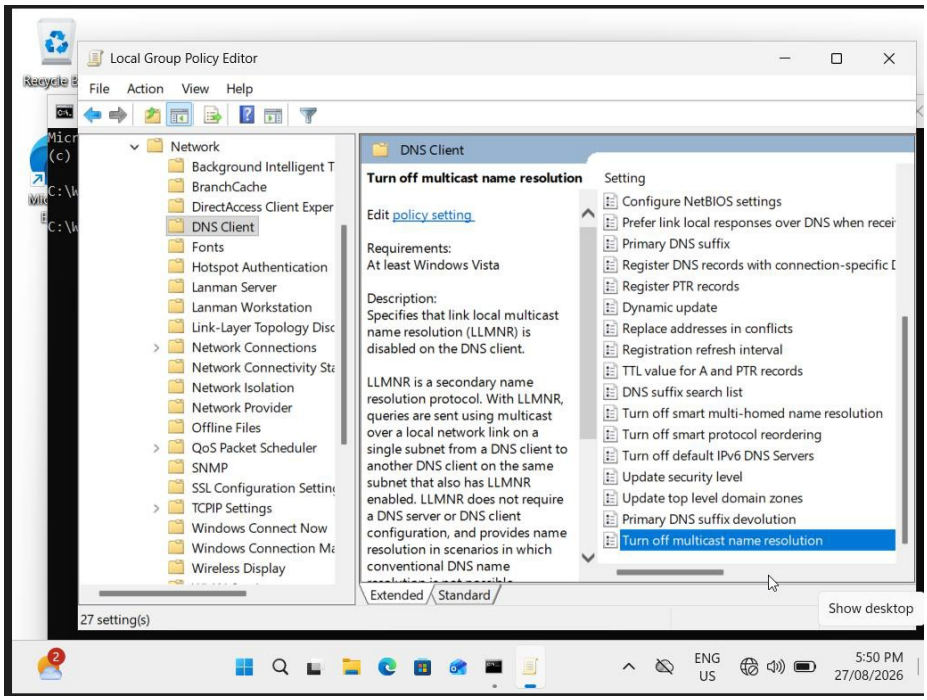
Phase 4 · Defense — Disabling LLMNR

With the attack proven, the next goal was to actually stop it — starting with the protocol used in the successful attack, LLMNR.

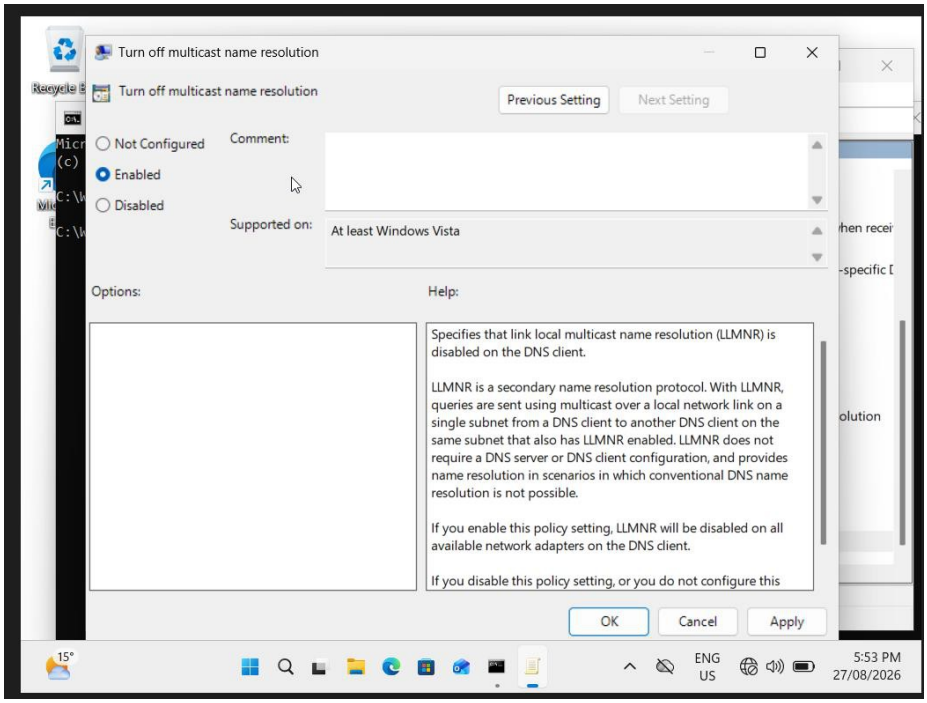
11. Opened the Local Group Policy Editor (gpedit.msc) on Windows 11 to find the setting that disables LLMNR.



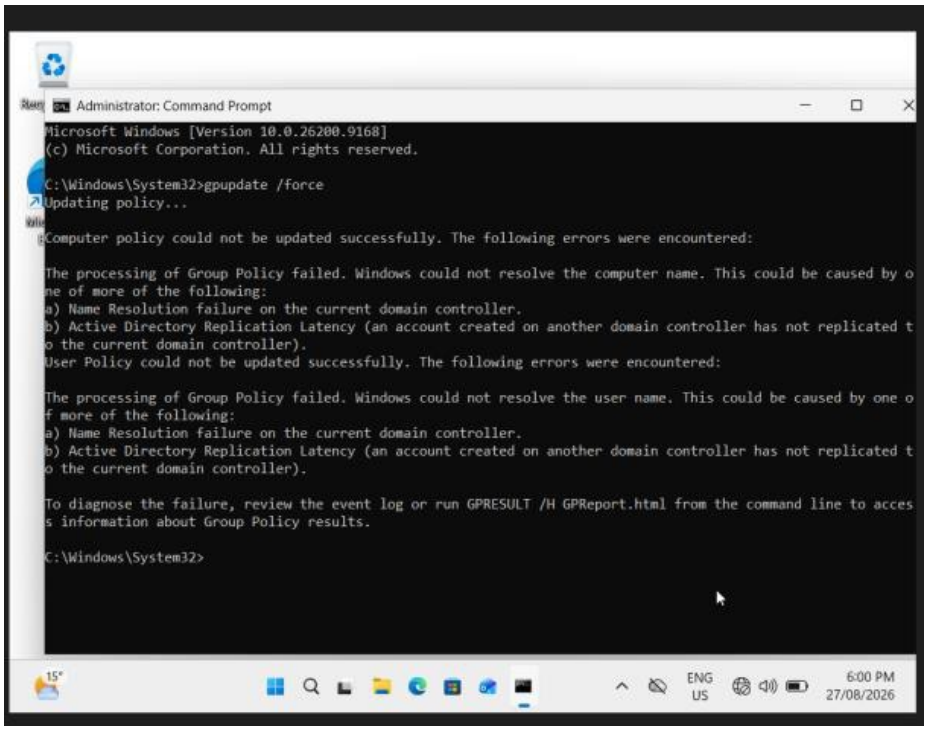
12. Navigated to Computer Configuration → Administrative Templates → Network → DNS Client, which is where the LLMNR setting lives.



13. Set "Turn off multicast name resolution" to Enabled. This setting name is worded in the negative, so choosing Enabled is what actually disables LLMNR on Windows 11 — closing the exact vulnerability the earlier attack exploited.

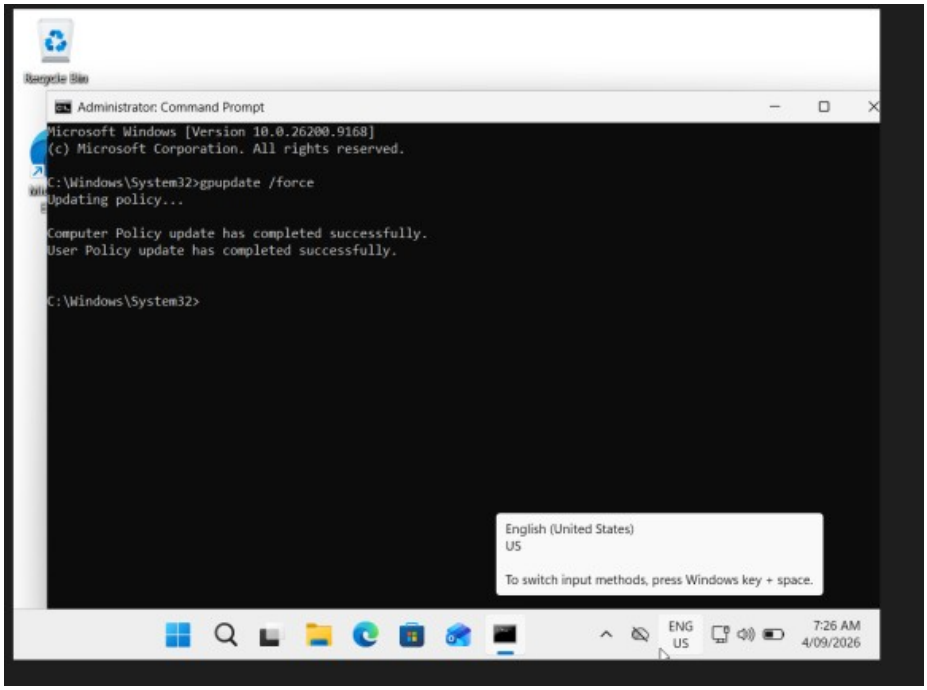


14. Ran gpupdate /force to apply the new policy immediately, but hit a real network error: Windows 11 couldn't reach the domain controller to complete a full policy refresh ("Windows could not resolve the computer name," a Name Resolution / Active Directory Replication error). This turned into its own multi-step troubleshooting detour, separate from the local Group Policy change itself.



> **📌 The troubleshooting story behind that failure**
>
> An earlier unexpected shutdown of the Windows Server 2022 VM had thrown its clock 8 days out of sync with Windows 11. Since Active Directory logins rely on Kerberos — which rejects authentication when the two machines' clocks disagree by more than about 5 minutes — this broke domain authentication with an "LDAP Bind function call failed" error. Fixing it took several steps: resyncing both VMs' clocks with w32tm, discovering the domain controller's DNS service was unreachable due to its network profile having reverted to "Public" after the crash (correcting it back to a trusted profile), and finally a clean restart of Windows 11 to clear out stale Kerberos tickets left over from the whole ordeal.

15. After resyncing the clocks, correcting the server's network profile, and restarting Windows 11, gpupdate /force finally completed with no errors: "Computer Policy update has completed successfully. User Policy update has completed successfully." This confirmed the LLMNR-disabling setting was now genuinely enforced by the domain, not just sitting unconfirmed locally.

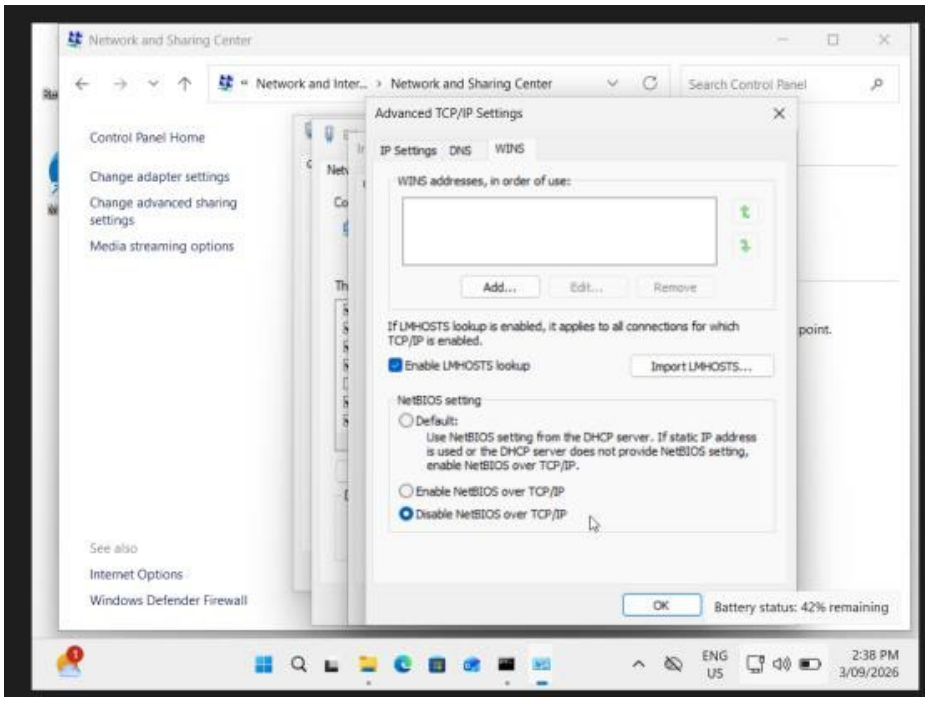


Phase 5 · Defense — Closing the NBT-NS and mDNS Gap

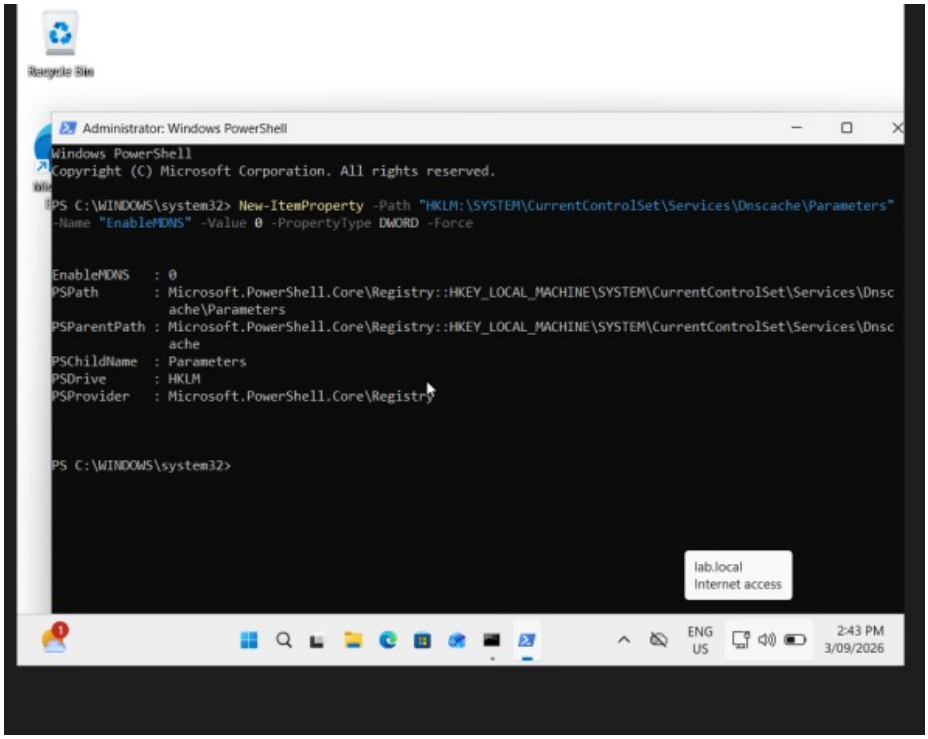
Retesting the attack after the LLMNR fix revealed the defense wasn't actually complete — a genuinely useful, realistic finding.

> **📌 The gap the first fix missed**
>
> Re-running the exact same attack after disabling LLMNR showed it still worked. Responder's terminal showed [MDNS] and [NBT-NS] poisoned answers instead of [LLMNR] — meaning Windows 11 tried LLMNR first, got no response (as expected, since it was now disabled), and then automatically fell back to two older, separate name-resolution protocols: NBT-NS (a legacy NetBIOS-based method) and mDNS (multicast DNS). Responder listens for all three by default, so it still captured a hash. This is a well-known, real gap in the security field — disabling LLMNR alone is not a complete fix.

16. Disabled NBT-NS through Windows 11's network adapter settings: Network and Sharing Center → Ethernet Properties → TCP/IPv4 → Advanced → WINS tab → "Disable NetBIOS over TCP/IP." This setting reverted itself once after a reboot (a known quirk caused by the DHCP lease renewing and re-pushing its own NetBIOS preference), so it had to be reapplied and immediately re-verified without another reboot.



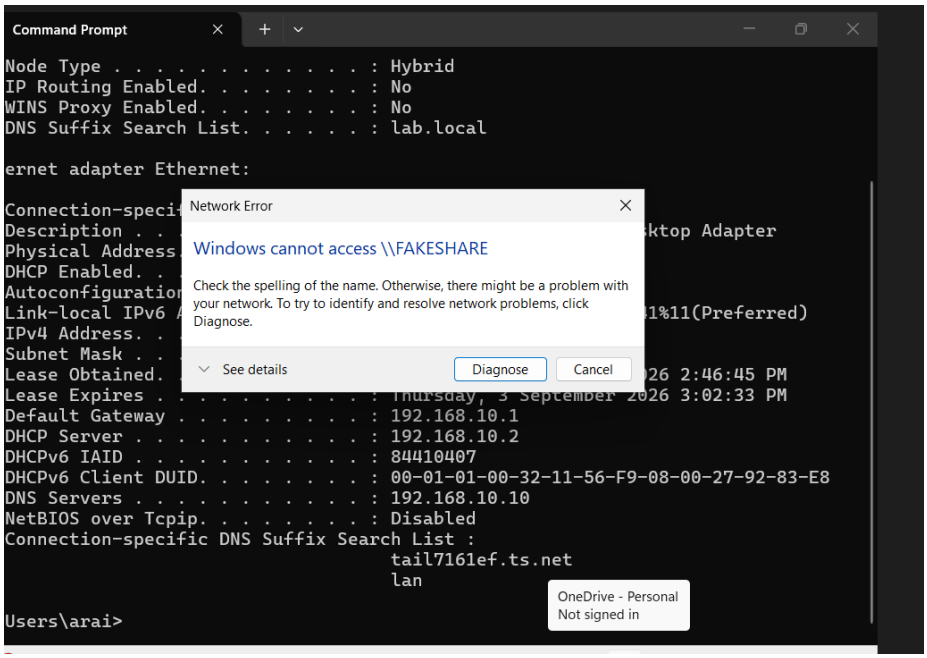
17. Disabled mDNS using a PowerShell command that sets the EnableMDNS registry value to 0 under the DNS Client service's settings — there is no checkbox for this anywhere in Windows' settings menus, so the registry is the only way to turn it off. The command's output confirmed EnableMDNS : 0, meaning the change applied successfully.



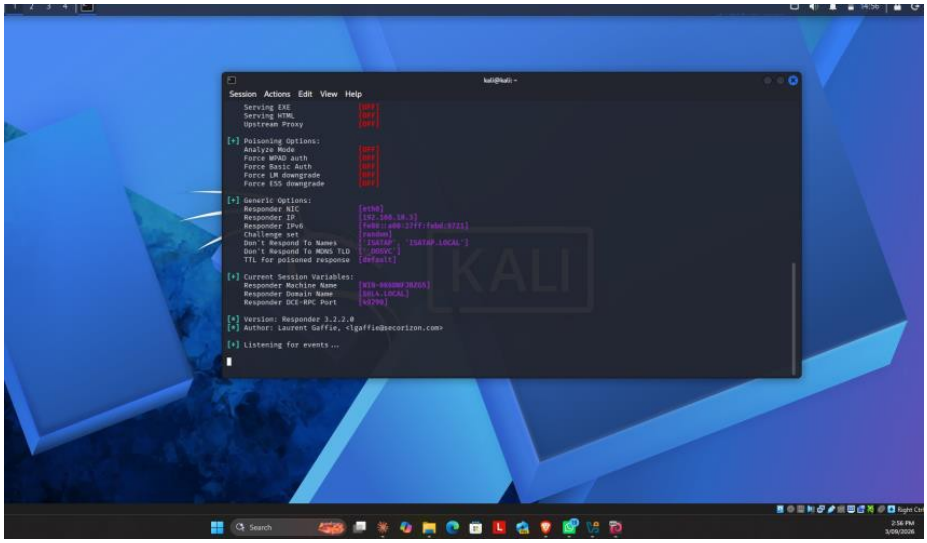
Phase 6 · Final Verification

With LLMNR, NBT-NS, and mDNS all disabled, the attack was run one final time to prove the fix actually holds.

18. On Windows 11, the same \\FAKESHARE lookup was triggered one more time. This time Windows simply returned "Windows cannot access \\FAKESHARE" — a clean failure, with no credential prompt and nothing sent over the network to leak.



19. On Kali, Responder was left listening throughout the retest. Its terminal stayed completely silent — no [LLMNR], [NBT-NS], or [MDNS] poisoned-answer lines, and no captured hash. Compared against the very first successful capture back in Phase 2, this is a clean, verified before-and-after result.



What this project demonstrates:

- A full offensive/defensive cycle: executing a real LLMNR poisoning attack, capturing a live NTLMv2 hash, and cracking it to a plaintext password.
- Practical problem-solving under real tool failures — a GPU-less Hashcat dead end, a corrupted hash file, and switching tools (John the Ripper) rather than getting stuck.

- Genuine Active Directory / Kerberos troubleshooting — diagnosing and fixing a clock-drift-driven authentication failure, including a stale network profile and a broken DNS lookup path.
- Recognizing an incomplete fix — discovering that disabling LLMNR alone left NBT-NS and mDNS as working fallback paths for the same attack, then closing both.
- Verifying the fix, not just assuming it — retesting the exact same attack after the full fix and confirming, side by side, that it no longer succeeds.

Tools used: Kali Linux, Responder, Hashcat, John the Ripper, VirtualBox, Windows Server 2022 (Active Directory), Windows 11, Group Policy, PowerShell.

This is a personal home lab project, not professional or client work — built while studying toward CompTIA Security+.